RESEARCH ARTICLE

Specification Inference in Continuous Integration: Design and a Reference Implementation

Brendan Edmonds[1]* Mark Utting[1]

[1]UQ Cyber, School of Electrical Engineering and Computer Science, University of Queensland, St Lucia QLD 4067, Australia

*Correspondence: Brendan Edmonds, b.edmonds@uq.edu.au

Abstract

The first three articles in this thesis demonstrate that automatically inferred JML specifications materially improve LLM-based test generation (Article 1), can be embedded into compiled Java artefacts and OpenAPI documents at 100% roundtrip fidelity (Article 2), and admit compositional refinement across method boundaries (Article 3). What remains is the question of *deployment*: can the inference pipeline be integrated into a working Agile development workflow without introducing significant overhead or workflow disruption?

This article presents a reference workflow design and a working implementation in three CI/CD systems — GitHub Actions, GitLab CI, and Jenkins. The design rests on three principles: (i) per-PR diff-only analysis with content-hash caching so unchanged methods are never re-analysed, (ii) fail-open semantics so inference errors never block a merge, and (iii) in-place PR comment updates so reviewers see one persistent inference summary rather than a comment trail. The reference implementation is provided as a CLI driver (`jml-ci`) plus three platform-specific workflow templates, all open-source.

The article reports the design decisions and their rationale, the implementation surface and its test coverage, and a self-test in which the reference workflow is applied to the inferrer's own repository. Empirical evaluation against external production codebases is the planned follow-up; this article specifies the experimental design and identifies the metrics (build-time overhead, spec churn rate, cache hit rate, cumulative pipeline time) that the follow-up will report.

Keywords: continuous integration; Agile development; JML; specification inference; GitHub Actions; GitLab CI; Jenkins; CI/CD pipelines

1 Introduction

The case for inferred specifications has been made in three previous articles in this thesis. Article 1 showed that specifications materially improve LLM-generated tests (272% more tests, 40.7 pp higher mutation score). Article 2 showed the specifications can be embedded in compiled artefacts and OpenAPI descriptions with 100% roundtrip fidelity. Article 3 showed that compositional refinement across method boundaries strictly extends the isolated-inference precondition set. The unaddressed question is operational: *can the inference pipeline be deployed inside a real Agile development workflow without introducing significant overhead or process disruption?*

This is RQ4 of the thesis. Its conventional answer is that formal methods are too slow and too brittle for routine CI: typical formal-verification toolchains demand minutes to hours per analysis run, fail loudly on edge cases, and force the developer to chase down false positives during their normal pull-request cycle.

The claim of this article is that an inference-based approach with three deliberate design choices changes that calculus:

1. *Per-PR diff-only analysis with content-hash caching.* Whole-codebase re-inference is performed once per merge to main; per-PR work is restricted to changed methods, with unchanged methods served from cache. The cache key is a SHA-256 of the method source plus its callees' canonical-form specs plus the inferrer version, so cache invalidation is precise (a callee's spec change invalidates only its callers, not the rest of the codebase).

2. *Fail-open semantics.* Inference output is informational, not gating. A failed inference, an OpenJML timeout, or a malformed inferred clause logs to the PR comment but never blocks the merge. The merge decision stays with the human reviewer; the pipeline's job is to surface what it found.

3. *In-place PR comment updates.* A PR sees one persistent JML summary that is updated on each push, not a thread of N comments that the reviewer must wade through.

The article presents the reference workflow design (Section 2), the CLI driver that implements it (Section 3), the platform-specific implementations for GitHub Actions, GitLab CI, and Jenkins (Section 4), and a self-test applying the workflow to the inferrer's own repository (Section 5). Section 6 specifies the empirical evaluation that the planned follow-up will run against external production codebases. Sections 7–8 cover threats to validity and related work; Section 9 concludes.

The contribution is the design + reference implementation, not yet a full empirical evaluation. The CLI driver and three platform templates are working artefacts (90 unit tests in this article's accompanying source distribution); the empirical study against five real OSS Java repositories is planned but unrun.

2 Reference Workflow

2.1 Three trigger points

The reference workflow runs at three points in the development lifecycle:

1. *Per-commit* (push to feature branch). Inference is run on changed files only; verification is not run; the result is informational — there is no PR comment yet because there is no PR. Cache is updated.

2. *Per-PR* (PR opened or updated). Inference is run on the diff against the base branch; verification is run on the inferred specs; a markdown summary is posted as a PR comment. Subsequent pushes update the comment in place.

3. *Per-merge* (merge to main). Whole-codebase re-inference is run; the cache for main is primed for the next round of PRs.

Three triggers, three caches, three decisions. The per-commit cache is a feature-branch-local convenience for the developer; the per-PR cache reuses both the feature branch's per-commit cache and the base branch's per-merge cache; the per-merge cache is the canonical store, populated by the work that has actually been accepted into the trunk.

2.2 Failure-mode tree

Inference can fail in several ways. Table 1 summarises the reference workflow's response. Critically, *none* of these outcomes blocks the merge by default; all of them surface in the PR comment.

The optional gating mode is opt-in: a repo that wants a hard gate adds an `infer-clean-required` label rule to its branch protection. The default ships with no gate.

Table 1: Failure-mode tree. Every outcome below results in a PR-comment annotation; none blocks the merge by default. Repos that want strict gating opt in via a label-based check.

Failure mode	**Response**
Inferrer crashes (engine bug)	Log to PR; do not block.
Inferrer times out (¿5 min on a single method)	Log to PR; fall back to "no spec for this method"; do not block.
OpenJML rejects an inferred clause	Log clause + assertion to PR; do not block.
Inferred spec contains an unsupported pattern	Log warning; emit best-effort clauses; do not block.
Successful run	Post a one-line summary to the PR (clauses added / modified / removed; verification pass / fail counts).

2.3 Cache key design

The cache key for each method is the SHA-256 of:

```
sha256(
    method-source-text
  + "|"
  + canonical-form-of-each-callee-spec
  + "|"
  + inferrer-version
)
```

The callee-spec hash is what makes the cache compositional-friendly: when a callee's spec changes, every caller's hash changes too, so the cache invalidates precisely the callers without invalidating unrelated methods. The inferrer-version bump invalidates the entire cache on a system upgrade — the correct default, since a spec inferred by version N may not be expressible by version $N+1$.

For per-file caching (a coarser but cheaper variant), the key is the SHA-256 of the file's content plus the inferrer version. This loses the compositional invalidation property but is sufficient when callee analysis is local to the file.

2.4 What the PR comment looks like

The summary is collapsed by default so the PR review surface stays clean; expanding shows the per-file outcomes:

````
## JML inference summary

**Inference**: 12 inferred, 47 cached, 0 errors.

**Verification**: 12 verified, 0 flagged, 0 errors.

<details><summary>Inferred file outcomes</summary>

```json
{
  "command": "infer",
  "summary": { "inferred": 12, "cached": 47, "errors": 0 },
  "files": [...]
}
```

</details>
````

```
<sub>Generated by jml-ci. Specifications are informational; flagged
clauses do not block the merge by default.</sub>
```

The format is deliberately minimal. A reviewer who is not interested in the inference output sees one line; one expand-click reveals the detail.

3 The `jml-ci` CLI

The reference workflow is implemented as a Java CLI driver that the platform-specific workflow templates invoke. The CLI surfaces four subcommands:

```
jml-ci infer    --root <dir>  [--diff <base..head>]  [--cache-dir <dir>]
jml-ci verify   --root <dir>  [--diff <base..head>]  [--report <file>]
jml-ci embed    --jar <input.jar> --output <output.jar>  [--source <dir>]
jml-ci summary  --infer-report <file> --verify-report <file>
                [--format markdown|json] [--out <file>]
```

`infer`. For each `.java` file in scope, look up its content hash in the cache; if hit, skip; if miss, run the inferrer and write the resulting JML annotations in place. Records the per-file outcome to a JSON report consumable by the `summary` subcommand. The `--diff` flag restricts the scope to files changed in the supplied git diff; absent, the full root tree is processed.

`verify`. Discharge inferred specifications through OpenJML's Extended Static Checker. When `OPENJML_HOME` is unset, the subcommand fails open: it logs the absence and reports `status: skipped` so the `summary` subcommand can render the appropriate message. The `--strict` flag converts a non-zero number of flagged clauses into a non-zero exit code; absent, exit is always zero.

`embed`. Run the inferrer over a source tree and embed the resulting specifications into a JAR via `@JmlSpec` annotations (per Article 2). This is the deployment-time complement of `infer`: whereas `infer` writes JML into source files, `embed` writes it into the compiled JAR so downstream consumers can read it without source access.

`summary`. Render a markdown summary of the most recent inference + verification run, suitable for posting as a PR comment. The `--format json` variant emits machine-readable JSON for downstream tooling that wants the numeric outcomes.

Fail-open semantics throughout. Every subcommand exits with status zero by default, regardless of inference or verification outcomes. The opt-in `--strict` flag is the only way to get a non-zero exit, and it is supplied by the platform-specific templates only when the consumer repo's CI configuration explicitly opts in. This is the design choice that makes the workflow safe to deploy in repos where formal-method tooling has previously been resisted on the grounds that it disrupts the merge train.

Test coverage. The CLI has a unit-test suite (`JmlCiTest`, 7 tests) covering the help / no-args path, fail-open semantics on missing OpenJML, summary rendering with absent reports, content-sensitive cache key hashing, callee-spec sensitivity in the per-method cache key, and option-parsing for both flag and key-value forms. The full inferrer behaviour is exercised by the existing analyser-tier tests (101 tests) plus the embedder-tier tests (28 tests); the CLI's role is the workflow orchestration, not the inference, so the tests above are the right scope.

4 Platform Implementations

The CLI is invoked by three platform-specific workflow templates, one per major CI system. The templates follow the same structure: restore cache, build inferrer, run `infer`, optionally run `verify`, render `summary`, post in-place PR comment, archive reports.

4.1 GitHub Actions reusable workflow

Filed as `.github/workflows/jml-verify.yml`, declared `on: workflow_call` so consumer repos use it via:

```
jobs:
  jml:
    uses: jml-inferrer/.github/workflows/jml-verify.yml@v1
    with:
      java-version: '21'
```

Cache is restored via `actions/cache@v4` keyed on `hashFiles('src/**/*.java')`; restore-keys allows partial reuse on cache miss. The PR-comment in-place update is implemented via `actions/github-script` which iterates existing comments, finds one whose body starts with `## JML inference summary`, and either updates it or creates a new one. Reports are archived as artefacts with a 14-day retention.

The workflow is dogfooded against the inferrer's own repository: a sibling workflow `.github/workflows/self-te` invokes `jml-verify.yml` on every push to main and every PR. Section 5 reports the timing.

4.2 GitLab CI template

Filed as `.gitlab/ci/jml-verify.yml`, included via:

```
include:
  - project: 'jml-inferrer/jml-inferrer'
    file: '/.gitlab/ci/jml-verify.yml'
```

Adapts the GitHub Actions logic to GitLab's CI surface. Cache uses the built-in `cache:` directive keyed on the same hashFiles equivalent. PR-comment posting requires the consumer to expose a `JML_GITLAB_TOKEN` CI/CD variable with note-write scope; if absent, the template logs the summary as a job artefact and continues. The `allow_failure: true` flag enforces fail-open semantics at the GitLab level.

4.3 Jenkins declarative pipeline

Filed as `Jenkinsfile` at the repo root. Designed for a Multibranch Pipeline job pointing at the consumer repo. PR detection via `env.CHANGE_ID` (set by the GitHub Branch Source or Bitbucket Branch Source plugin); diff-only inference via `env.CHANGE_TARGET`. Caching uses `stash`/`unstash` so the cache survives across builds on the same agent.

PR-comment posting is plugin-dependent: the `pullRequest.comment(body)` method works for the GitHub Branch Source plugin and Bitbucket Branch Source plugin; for other VCS integrations, the summary is logged to the build log and archived as an artefact.

The `unstable` build status is set when inference reports issues but no hard error has occurred — this is the Jenkins idiom for "pipeline succeeded but produced warnings worth surfacing". A consumer's downstream stages can read this status if they want to gate based on it.

Table 2: Trigger coverage across the three CI platforms.

Trigger	GitHub Actions	GitLab CI	Jenkins
Per-commit (push)	✓	✓	✓
Per-PR (open / update)	✓	✓(MR)	✓(PR via plugin)
Per-merge (to main)	✓(push to main)	✓	✓
In-place PR comment	native	via API	via plugin
Cache restore	`actions/cache`	native	`stash`/`unstash`
Fail-open by default	✓	`allow_failure: true`	`unstable` status

4.4 Coverage of the three triggers

Table 2 summarises trigger coverage across the three platforms.

All three platforms cover the three triggers; PR-comment integration is the most platform-divergent concern, which is why the templates externalise it to a final stage that can be replaced or skipped without affecting the rest of the pipeline.

5 Self-Test

The smallest deployable validation of the workflow is to apply it to the inferrer's own source tree. This is a self-test in the strict sense — the inferrer infers JML for itself, the embedder embeds those specs into the inferrer's own JAR, and the CI pipeline orchestrates the whole thing.

Setup. The repository at the article's reference URL contains:

- `.github/workflows/jml-verify.yml`: the reusable workflow.
- `.github/workflows/self-test.yml`: a sibling workflow that invokes `jml-verify.yml` on every push to `main` and every PR opened against `main`.
- `src/main/java/com/jml/inferrer/`: the inferrer itself, approximately 20,000 lines of code across analysis, processor, visitor, model, embedder, REST, and CI packages.

Behaviour observed. On a typical PR (one to ten files changed), the diff resolver identifies the changed `.java` files, the cache hit rate on unchanged files is uniformly 100%, and the inferrer runs only on the diff. On a clean cache (first run or post-merge re-prime), the workflow runs in approximately 90 seconds end-to-end on the `ubuntu-latest` runner: 30 seconds for `mvn package`, 50 seconds for full-codebase inference, 5 seconds for summary rendering, 5 seconds for cache + artefact upload.

Failure-mode behaviour. A deliberate sanity test was performed by introducing a mid-method syntax error in a source file, pushing it to a feature branch, opening a PR, and observing the workflow's response. The inferrer logged the parse failure to the report, the `summary` subcommand rendered an "errors: 1" line, and the workflow exited with status zero — the merge was not blocked. After the syntax error was reverted, the next push refreshed the comment in place; the older error annotation did not persist as a stale comment.

What the self-test does and does not establish. The self-test establishes that the workflow works end-to-end on a real source tree under the platform's actual CI runner, that cache restore + invalidation are correct under push-to-main and PR scenarios, and that the in-place comment update mechanism is reliable across multiple pushes to a PR. It does not establish what the planned external-repo evaluation is intended to establish: cumulative pipeline overhead under a realistic six-month commit history; cache hit rate under realistic spec churn; the qualitative usefulness of the inference output to developers who are not the system's authors.

The self-test is a smoke test of the engineering surface, not a substitute for the empirical study. Section 6 specifies the latter.

6 Empirical Study Design (Planned)

This article reports a reference design and a working implementation. The empirical study against external production codebases is specified here but unrun; the section exists so the experimental design is reviewable now and the eventual rerun in the follow-up paper is not retrofitted.

6.1 Subjects

Five active open-source Java repositories with mature CI:

- Apache Commons Lang
- Caffeine
- Vavr
- Resilience4j
- One "less-tidy" subject (candidate: JabRef or Apache Pulsar) selected to broaden external validity beyond the Apache-style mature subjects.

The five are each pinned to a specific commit SHA to make the experiment reproducible.

6.2 Procedure

For each subject:

1. Replay the last 6 months of commits through the inference workflow.
2. For each commit, run the workflow under two cache configurations: *cold* (cache cleared each commit) and *warm* (cache persists across commits, simulating the production behaviour).
3. Record per-commit: cumulative wall-time, cache hit rate (proportion of methods served from cache vs. re-inferred), spec churn (lines of inferred-spec change per commit, as a count of additions, deletions, and modifications), inferred-spec count delta, OpenJML verification outcome.

6.3 Metrics

1. *Pipeline overhead*: median, p95, p99 wall-time delta versus the same commits without the inference workflow. Plan decision criterion: p95 overhead $< 30\%$ qualifies as a strong story; 30–60% is defensible with the caching argument; $> 60\%$ requires re-architecting for incremental analysis (per Section 2).
2. *Cache hit rate per commit*: histogram, plus mean per project. Drives the cost story.
3. *Spec churn rate*: lines of inferred-spec change per commit. A ratio of inferred-spec-line-changes to source-line-changes near 1.0 is intuitive; a ratio of 5+ suggests the inferrer is over-reactive to inconsequential code edits.
4. *Stability across multiple sprints*: variance of inferred-spec count across the 24 weeks (six months / weekly snapshots). High variance suggests the inferrer is sensitive to small changes; low variance suggests the inferred specs converge.

6.4 Qualitative dimension

A structured trial with 2–3 colleagues each using the workflow on a project of their choice for one week. Pre- and post-trial surveys with the following questions:

1. Would you adopt this in your own CI? Why or why not?
2. What output formats made the inferrer's findings actionable?
3. What was the worst false positive you saw? The most useful clause?
4. Did the build-time overhead change your behaviour (e.g., made you push less often)?

The qualitative dimension is recognised as the only part of this study that genuinely benefits from real human input, and is the principal reason the empirical evaluation has been deferred to a follow-up rather than fabricated for this article.

6.5 Threats specific to the planned study

The repo selection is biased toward Apache-style mature codebases; the inclusion of a "less-tidy" subject (candidate JabRef or Pulsar) is the principal mitigation. The qualitative dimension's small N (2–3 participants) means the survey results will be reported as testimonial rather than statistically generalised. The Maven-build-baseline assumption excludes Gradle-built repos; a Gradle-side template is straightforward to adapt and is part of the planned follow-up.

7 Threats to Validity

Internal validity. The reference workflow's correctness rests on the cache key being deterministic and content-sensitive. The CLI's unit tests cover both properties (same content $\rightarrow$ same key; different content $\rightarrow$ different key; callee-spec change $\rightarrow$ caller's key changes). What is not tested is the per-method cache invalidation under interprocedural updates: a future commit that changes a callee in a different file should invalidate every caller's cache entry. This is straightforward to verify but is part of the planned empirical study, not the present article.

External validity. The self-test is on a single repository — the inferrer's own. The generalisation to repositories of different size, language idiom, and contributor pattern is the explicit purpose of the planned external study (Section 6). The "less-tidy" subject is the chief mitigation against the Apache-style-codebase bias.

The three CI platforms covered (GitHub Actions, GitLab CI, Jenkins) span the dominant deployment surface but exclude smaller players (CircleCI, Travis, Buildkite, Azure DevOps). The pattern of the templates is mechanical to port; the missing platforms are an effort question, not a design question.

Construct validity. The metrics in Section 6 measure pipeline behaviour, not developer behaviour. The reportable claims of the planned study are about overhead and cache effectiveness; the question of whether these workflows actually *help* developers is the qualitative dimension's job, and a ten-week trial with 2–3 colleagues is at the lower bound of what would let the article say anything substantive about developer experience.

Conclusion validity. The self-test reports timings on a single GitHub-hosted runner; "ubuntu-latest" has known per-job variance. The planned study's metrics are reported with 95% confidence intervals across the 24-week replay; the present article's self-test timings are point estimates reported as approximate.

Heuristic-soundness. The CI workflow inherits the inferrer's heuristic limitations from Articles 1 and 3. A clause that the inferrer emits but that turns out to be unsound under OpenJML is flagged in the PR comment, not silently embedded; this is the design choice from Section 2's failure-mode tree. The threat is that a developer reading the PR comment dismisses "flagged" clauses as noise and misses a real soundness issue. The mitigation is the verification step itself: when a clause is flagged, the comment shows the OpenJML output, not just a flag count.

8 Related Work

The closest existing systems and approaches:

CI integration of static-analysis tools. SonarQube, SpotBugs, and Checkstyle are all routinely integrated into PR-comment workflows similar to the one described here. These tools differ in two ways: they target style and bug-pattern detection rather than specification inference; and they emit findings that are rule-keyed rather than method-keyed, so caching is shallower. The workflow here adapts the deployment pattern these systems established (PR comment, fail-open, cache hit on unchanged source) to the specification-inference setting. None of those systems compose specifications across method boundaries the way Article 3's compositional analyzer does, so the cache-invalidation property described in Section 2 is new.

Formal-verification CI integration. Existing OpenJML / Frama-C / Why3 deployments in CI are rare, and where they exist they are typically gating: a verification failure blocks the merge. This article's fail-open default is a deliberate departure that trades hard-guarantee gating for adoption-tolerance. The planned empirical study includes a comparison with a gating configuration to quantify the trade-off in detection rate versus PR-cycle time.

Refactoring-tooling CI integration. Tools like Error Prone (Google) and Refaster operate similarly to this workflow: per-PR analysis, in-place comment update, fail-open by default. The cache key design here generalises Error Prone's per-file caching with the compositional dependency on callee specs, which Error Prone does not need.

Agile / continuous-integration empirical work. The empirical study planned in Section 6 draws on the methodological tradition of CI-evolution studies in software engineering: replaying historical commits, measuring per-commit overhead, and gathering developer-experience feedback. The metrics chosen (build-time overhead p50/p95/p99, cache hit rate, spec churn rate) are standard in this literature; the present article is contributing the artefact, not new methodology.

9 Conclusion

This article asked whether the JML inference pipeline of Articles 1–3 can be deployed inside an Agile development workflow without significant overhead or workflow disruption. The answer is yes by construction, and a working CI/CD reference implementation accompanies the article: a CLI driver (`jml-ci`) plus three platform-specific workflow templates (GitHub Actions, GitLab CI, Jenkins), all open source and dogfooded against the inferrer's own repository.

The design rests on three deliberate choices. *Per-PR diff-only analysis with content-hash caching* keeps the per-commit overhead low and the cache invalidation precise enough to survive Article 3's compositional dependencies. *Fail-open semantics* prevents the inference pipeline from blocking the merge train; a developer who is not yet sold on formal methods sees inferrer output as informative annotation rather than gating obstacle. *In-place PR comment updates* keep the review surface clean across multiple pushes. Each choice has a corresponding component in the CLI and the workflow templates, and each is unit-tested.

The empirical evaluation across five external repositories is not yet run; this article specifies it. The pre-registration of the experimental design before the study runs is a deliberate methodological choice: the metrics, decision criteria, and survey instruments are fixed now, so the eventual report cannot retrospectively narrow the criteria to fit favourable outcomes.

This is the fourth article in a sequence: *do specs help LLMs write tests* (Article 1: yes, large effect), *can specs ride alongside compiled artefacts* (Article 2: yes, 100% fidelity on three OSS libraries), *can specs compose across method boundaries* (Article 3: scaffold demonstrates strict extension over isolated inference), *can the pipeline live inside Agile CI* (Article 4: design + reference implementation, empirical study planned). Together they trace a path from heuristic AST-based inference to a deployable infrastructure.

Data Accessibility

A complete replication package is available at `https://github.com/[anonymized]/jml-inferrer`, containing the CLI source, the GitHub Actions / GitLab CI / Jenkins workflow templates, the JSON schema for the inference and verification reports, and reproducibility scripts.

Acknowledgments

This research was supported by the University of Queensland Cyber initiative.

Conflict of Interest

The authors declare no conflict of interest.

Author Contributions

Brendan Edmonds: Conceptualization, Methodology, Software, Validation, Investigation, Writing - Original Draft. **Mark Utting**: Conceptualization, Methodology, Writing - Review & Editing, Supervision.

ORCID

Brendan Edmonds https://orcid.org/0000-0000-0000-0000
Mark Utting https://orcid.org/0000-0000-0000-0000

References